

Stored Procedures

Reusable Server-Side Business Logic in SQL

■ LEARNING OUTCOME

ss logic in

■ Skills You Will Gain

- Create and call stored procedures with parameters
- Write user-defined functions that return values
- Use IF/ELSE, LOOP, WHILE control flow in procedures
- Implement transactions with COMMIT and ROLLBACK
- Create triggers for automatic data validation
- Create and use views for reusable queries

■ **Stored procedures move business logic close to the data. They improve performance (less network round trips), security (users call procedures, not tables), and reusability across applications.**

SECTION 1

Stored Procedures

```
-- Create a stored procedure (MySQL syntax) DELIMITER // CREATE PROCEDURE GetCustomerOrders( IN
p_customer_id INT, IN p_status VARCHAR(20), OUT p_total_orders INT ) BEGIN -- Declare variables
DECLARE v_customer_name VARCHAR(100); -- Get customer name SELECT CONCAT(first_name, ' ',
last_name) INTO v_customer_name FROM customers WHERE customer_id = p_customer_id; -- Return
matching orders SELECT order_id, order_date, total_amount, status FROM orders WHERE customer_id =
p_customer_id AND (p_status IS NULL OR status = p_status) ORDER BY order_date DESC; -- Set the OUT
parameter SELECT COUNT(*) INTO p_total_orders FROM orders WHERE customer_id = p_customer_id; END //
DELIMITER ; -- Call the procedure CALL GetCustomerOrders(42, 'delivered', @total); SELECT @total AS
order_count; -- PostgreSQL procedure syntax CREATE OR REPLACE PROCEDURE
update_customer_tier(p_customer_id INT) LANGUAGE plpgsql AS $$ DECLARE v_spend DECIMAL; BEGIN
SELECT annual_spend INTO v_spend FROM customers WHERE customer_id = p_customer_id; UPDATE customers
SET is_premium = (v_spend > 100000) WHERE customer_id = p_customer_id; COMMIT; END; $$;
```

SECTION 2

Control Flow and Error Handling

```
-- IF / ELSEIF / ELSE DELIMITER // CREATE PROCEDURE AssignTier(IN p_spend DECIMAL) BEGIN DECLARE
v_tier VARCHAR(20); IF p_spend >= 200000 THEN SET v_tier = 'Platinum'; ELSEIF p_spend >= 100000
THEN SET v_tier = 'Gold'; ELSEIF p_spend >= 50000 THEN SET v_tier = 'Silver'; ELSE SET v_tier =
'Bronze'; END IF; SELECT v_tier AS customer_tier; END // DELIMITER ; -- WHILE loop DELIMITER //
CREATE PROCEDURE GenerateMonthlyReport(IN p_year INT) BEGIN DECLARE v_month INT DEFAULT 1; WHILE
v_month <= 12 DO INSERT INTO monthly_reports (yr, mo, revenue) SELECT p_year, v_month,
SUM(total_amount) FROM orders WHERE YEAR(order_date) = p_year AND MONTH(order_date) = v_month; SET
v_month = v_month + 1; END WHILE; END // DELIMITER ; -- Error handling with DECLARE HANDLER
DELIMITER // CREATE PROCEDURE SafeInsertOrder( IN p_customer_id INT, IN p_amount DECIMAL ) BEGIN
DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; SELECT 'Error occurred, transaction rolled
back' AS error_msg; END; START TRANSACTION; INSERT INTO orders (customer_id, total_amount,
order_date, status) VALUES (p_customer_id, p_amount, CURRENT_DATE, 'pending'); UPDATE customers SET
annual_spend = annual_spend + p_amount WHERE customer_id = p_customer_id; COMMIT; END // DELIMITER
;
```

SECTION 3

Functions, Views, and Triggers

```
-- USER-DEFINED FUNCTION: returns a single value, usable in SELECT CREATE FUNCTION
GetCustomerTier(p_spend DECIMAL) RETURNS VARCHAR(20) DETERMINISTIC BEGIN RETURN CASE WHEN p_spend
>= 200000 THEN 'Platinum' WHEN p_spend >= 100000 THEN 'Gold' WHEN p_spend >= 50000 THEN 'Silver'
ELSE 'Bronze' END; END; -- Use the function in a query SELECT first_name, annual_spend,
GetCustomerTier(annual_spend) AS tier FROM customers; -- VIEW: virtual table built from a SELECT
CREATE VIEW vw_customer_summary AS SELECT c.customer_id, c.first_name, c.last_name, c.city,
COUNT(o.order_id) AS total_orders, SUM(o.total_amount) AS lifetime_value, MAX(o.order_date) AS
last_order_date, GetCustomerTier(SUM(o.total_amount)) AS tier FROM customers c LEFT JOIN orders o
ON c.customer_id = o.customer_id GROUP BY c.customer_id, c.first_name, c.last_name, c.city; --
Query a view just like a table SELECT * FROM vw_customer_summary WHERE tier = 'Gold'; -- TRIGGER:
auto-execute on INSERT/UPDATE/DELETE CREATE TRIGGER trg_update_spend AFTER INSERT ON orders FOR
EACH ROW BEGIN UPDATE customers SET annual_spend = annual_spend + NEW.total_amount, last_order_date
= NEW.order_date WHERE customer_id = NEW.customer_id; END; -- NEW.column = new value
(INSERT/UPDATE) -- OLD.column = previous value (UPDATE/DELETE) -- Audit trigger: log all changes
CREATE TRIGGER trg_audit_orders AFTER UPDATE ON orders FOR EACH ROW BEGIN INSERT INTO
order_audit_log (order_id, old_status, new_status, changed_at) VALUES (OLD.order_id, OLD.status,
NEW.status, NOW()); END;
```

■ PRACTICE Q & A

Q1. What is the difference between a stored procedure and a function?

*A: Procedure: can have IN/OUT/INOUT params, returns zero or more result sets, called with CALL, cannot be used in SELECT.
Function: returns exactly ONE value, can be used in SELECT/WHERE/expressions, typically no side effects.*

Q2. What is the difference between a view and a table?

A: Table: stores actual data. View: virtual table -- stores only the SELECT query definition, no data. Every time you query a view, the underlying query executes. Views simplify complex queries and restrict column access.

Q3. What is a trigger and when would you use one?

A: A trigger auto-executes on INSERT, UPDATE, or DELETE. Use for: audit logging, denormalized counter updates, validation that enforces complex business rules, cascading updates across tables.

Q4. What is the difference between COMMIT and ROLLBACK?

A: COMMIT permanently saves all changes in the current transaction. ROLLBACK undoes all changes back to the last COMMIT or transaction start. Use with START TRANSACTION for atomic multi-step operations.

Q5. What does DETERMINISTIC mean in a function?

A: DETERMINISTIC means the function always returns the same output for the same inputs (no randomness, no time-based results). Required for functions used in indexes or replication.

■ Stored Procedures Cheat Sheet	
<code>CREATE PROCEDURE name(IN p INT)</code>	Stored procedure with input
<code>CALL procedure_name(42)</code>	Execute procedure
<code>OUT parameter</code>	Return value from procedure
<code>IF x THEN ... ELSEIF y THEN ... END IF</code>	Conditional logic
<code>WHILE cond DO ... END WHILE</code>	Loop in procedure
<code>START TRANSACTION / COMMIT</code>	Atomic multi-step operation
<code>ROLLBACK</code>	Undo transaction on error
<code>DECLARE EXIT HANDLER FOR SQLEXCEPTION</code>	Error handling
<code>CREATE FUNCTION name() RETURNS type</code>	User-defined function
<code>DETERMINISTIC</code>	Same input always same output
<code>CREATE VIEW name AS SELECT ...</code>	Virtual reusable query
<code>CREATE TRIGGER name AFTER INSERT ON tbl</code>	Auto-execute on data change